

GPT-Style Transformers

Engineering revision notes for reconstructing a decoder-only Transformer forward pass

Designed as a dense, notebook-ready reference: what each component is, why it exists, how it works, and what it feeds next.

Reading convention

Learned parameters are persistent tensors updated during training and frozen during inference. Activations are temporary, input-dependent tensors. Shapes use B=batch size, T=sequence length, V=vocabulary size, d=hidden size.

Convention used throughout: batch size B ; sequence length T ; vocabulary size V ; hidden size d ; per-head key/query size d_k ; feed-forward width d_{ff} ; h attention heads. GPT is a **decoder-only**, causal Transformer. Learned parameters change only during training; activations are temporary values recomputed from the input. In inference, every learned parameter is frozen.

1. High-Level GPT Pipeline

```

Input text
  ↓ tokenizer
token IDs  $x \in \{0, \dots, V-1\}^{(B \times T)}$ 
  ↓ embedding lookup
token embeddings  $E[x] \in \mathbb{R}^{(B \times T \times d)}$ 
  + positional embeddings  $P$ 
hidden states  $H \in \mathbb{R}^{(B \times T \times d)}$ 
  ↓ Transformer layer 1
 $H^1$  ↓ Transformer layer 2 ↓ ... ↓ Transformer layer  $L$ 
final hidden state of last position  $h_{\text{last}} \in \mathbb{R}^d$ 
  ↓ LM head
logits  $z \in \mathbb{R}^V \rightarrow \text{softmax} \rightarrow \text{next-token distribution } p \in \mathbb{R}^V$ 
  ↓ select token → append it → repeat
  
```

- **What:** A function that maps a token prefix to a distribution over the next token.
- **Why:** Text generation is decomposed into a tractable repeated task: “given all previous tokens, predict one more.”
- **How:** Transform every token representation through L causal layers; read only the final position to predict the continuation.
- **Next:** The sampled/selected ID is appended to the prefix, so it becomes context for the next pass.

2. Training vs Inference

- **Training:** feed a known sequence, predict each next-token target, compute cross-entropy loss, backpropagate, update parameters.
- **Inference:** feed a prefix, compute only the next-token distribution, select an ID, append it. No loss, no gradients, no parameter updates.
- **Key distinction:** parameters are persistent learned tensors; activations are input-dependent temporary tensors.

Component	Training	Inference
Embedding matrix $E \in \mathbb{R}^{(V \times d)}$	learned/updated	frozen; rows looked up
Positional embeddings $P \in \mathbb{R}^{(T_{\text{max}} \times d)}$	learned/updated (if learned)	frozen; rows looked up
$WQ, WK, WV \in \mathbb{R}^{(d \times d_k)}$ per head	learned/updated	frozen; multiply activations
$WO \in \mathbb{R}^{(h \cdot d_k \times d)}$	learned/updated	frozen
Feed-forward weights	learned/updated	frozen
Feed-forward biases	learned/updated	frozen
LayerNorm $\gamma, \beta \in \mathbb{R}^d$	learned/updated	frozen
LM head $W_{LM} \in \mathbb{R}^{(d \times V)}$	learned/updated	frozen
Biases, if present	learned/updated	frozen
Hidden states	computed; retained for gradients	computed; discarded after use (except needed cache)
Queries / keys / values	computed activations	computed activations; K/V may be cached
Attention scores / weights / outputs	computed activations	computed activations

Component	Training	Inference
Logits / probabilities	computed for loss	computed for selection

3. Vocabulary

- **Vocabulary:** fixed table of V token pieces known by the model.
- **Vocabulary size V :** number of possible output classes and rows in E ; e.g. 50k–200k.
- **Tokens:** subword pieces, words, punctuation, whitespace fragments, or bytes—not necessarily words.
- **Token ID:** integer index into the vocabulary; it has no numerical meaning or distance relation.
- **Why tokens, not words:** open-ended language makes a word-only vocabulary too large and cannot cleanly cover rare words, spelling, code, or multilingual text. Subword/byte tokenization gives finite coverage and reusable pieces.

4. Tokenization

"The cat sat" $\rightarrow$ ["The", " cat", " sat"] $\rightarrow$ [42, 317, 901]

- **What:** deterministic mapping between text and token IDs (typically BPE/Unigram/byte-based).
- **Why:** neural networks consume numbers; a compact reusable vocabulary prevents one class per possible word.
- **How:** tokenizer chooses vocabulary pieces that concatenate back to the original text; decoder reverses IDs to text.
- **Next:** IDs index rows of the embedding matrix.

5. Embedding Matrix

- **Embedding matrix $E \in \mathbb{R}^{(V \times d)}$:** learned parameter; row $E[i]$ is the d -dimensional representation for token ID i .
- **Input embeddings $X = E[x] \in \mathbb{R}^{(B \times T \times d)}$:** activation formed by selecting rows. E is one shared table; X varies with the prompt.
- **Why embeddings exist:** one-hot $e_i \in \mathbb{R}^V$ is sparse, huge, and cannot directly encode learned similarity. $e_i \blacksquare E = E[i]$ turns a discrete symbol into a dense working vector.
- **Inference:** E stays fixed. Input embeddings change only because different token IDs select different fixed rows.
- **Next:** add positional information to make H .

6. Positional Encoding

- **Position embedding $P[t] \in \mathbb{R}^d$:** learned parameter row (or a fixed sinusoidal/rotary-derived positional signal) for position t .
- **Initial hidden state $H[:, t, :] = X[:, t, :] + P[t] \in \mathbb{R}^d$:** activation.
- **Why:** self-attention alone is permutation-equivariant: swapping token positions swaps outputs, with no inherent “first/after/before” concept. Language order must be supplied.
- **Embedding answers:** “which token is this?” Positional signal answers: “where is it / how far apart are tokens?”
- **Next:** H is the first layer’s input.

7. Hidden States

- **Hidden state** $H_{\text{pos}} \in \mathbb{R}^{(B \times T \times d)}$: activation after layer l ; one d -vector per position. H_{pos} is embedding + position.
- **What it represents**: the model's current contextual interpretation of each token, given only allowed earlier tokens.
- **Why it exists**: token IDs are discrete labels. Every layer needs a continuous workspace where context can be read, transformed, and preserved.
- **Embedding vs hidden state**: embedding is context-free lookup; hidden state is context-dependent and layer-specific.
- **Attention output vs hidden state**: attention output is only the context-read contribution of one sublayer. After projection, residual, normalization, and FFN, it becomes part of the new hidden state.
- **Why richer per layer**: earlier layers often form local/syntactic features; later layers can compose features already mixed across context. No single layer is required to have a human-readable meaning.
- **Invariant**: hidden states are the **only** per-token vectors passed from one Transformer layer to the next. $Q/K/V$, scores, and attention outputs are intermediate activations and disappear after that layer (except cached K/V at inference).

8. Complete Transformer Layer

Post-LayerNorm form, matching this note's worked example (many modern GPTs use **pre-LN**: normalize before each sublayer instead):

$H_{\text{pos}}^1 \rightarrow Q, K, V \rightarrow \text{causal multi-head attention} \rightarrow W_O \rightarrow + H_{\text{pos}}^1 \rightarrow \text{LayerNorm}$
 $\rightarrow \text{Feed Forward} \rightarrow + \text{previous result} \rightarrow \text{LayerNorm} \rightarrow H_{\text{pos}}^2$

- **Why two sublayers**: attention communicates across positions; FFN computes a nonlinear per-position transformation.
- **Next**: H_{pos}^2 feeds the next identical-structure layer with different parameters.

9. W_Q, W_K, W_V, W_O

For one head, with input $H \in \mathbb{R}^{(B \times T \times d)}$:

Tensor	Shape	Type	Purpose / next use
W_Q	$d \times d_k$	learned	makes queries $Q = HW_Q$
W_K	$d \times d_k$	learned	makes keys $K = HW_K$
W_V	$d \times d_v$	learned	makes values $V = HW_V$ (usually $d_v = d_k$)
W_O	$(h \cdot d_v) \times d$	learned	mixes concatenated head outputs back to hidden width

- **Why three projections**: “where to look” (Q), “how to be found” (K), and “what content to send” (V) are different jobs. Forcing one vector to do all three reduces expressive freedom.
- **Why each layer differs**: each layer performs a new learned computation over a differently contextualized H ; sharing would repeat the same operation.
- **W_O vs W_{LM}** : W_O maps attention features back into hidden space d ; W_{LM} maps final hidden space to vocabulary V .

10. Query, Key, Value

For a head: $Q, K \in \mathbb{R}^{(B \times T \times d_k)}, V \in \mathbb{R}^{(B \times T \times d_v)}$ are temporary activations.

- **Q** : “What information am I looking for?” A query at destination position i is compared with all allowed keys.
- **K** : “What kind of information do I contain?” A key at source position j determines whether j matches a query.
- **V** : “What information will I contribute if selected?”

- **Why compare Q with K:** dot products give learned compatibility, separately for every destination/source pair.
- **Why never compare V:** a value is payload, not addressing metadata. It is mixed only after addressing weights are decided.

11. Attention

For one head and one batch item, $Q, K \in \mathbb{R}^{(T \times d_k)}$, $V \in \mathbb{R}^{(T \times d_v)}$:

$QK^T \rightarrow$ raw scores $S \in \mathbb{R}^{(T \times T)} \rightarrow$ add causal mask $\rightarrow$ divide by $\sqrt{d_k}$
 $\rightarrow$ row-wise softmax $A \in \mathbb{R}^{(T \times T)} \rightarrow AV \rightarrow O \in \mathbb{R}^{(T \times d_v)}$

$S_{ij} = q_i \cdot k_j$. Position i is the query/destination; j is key/value/source.

- **Dot product / similarity:** high when learned query and key features align; “similarity” means learned relevance, not necessarily semantic similarity.
- **Scores S :** unnormalized compatibility numbers; can be negative and do not sum to one.
- **Weights A :** nonnegative normalized scores; each row sums to 1 over allowed sources.
- **Output $O_i = \sum_j A_{ij} v_j$:** context vector gathered for position i .
- **Next:** concatenate head outputs, project with W_O , then residual path.

12. Causal Mask

- **Mask $M \in \mathbb{R}^{(T \times T)}$:** fixed non-learned tensor, $M_{ij}=0$ when $j \leq i$, $-\infty$ when $j > i$.
- **Why:** GPT is decoder-only and must predict token $i+1$ using positions $\leq i$. Allowing future positions during training leaks the answer (“cheating”) and breaks generation.
- **Applied:** $S_{\text{masked}} = S + M$ before softmax; $\text{softmax}(-\infty)=0$.
- **Next:** each row can weight only its past and itself.

13. Scaling

$S_{\text{scaled}} = S_{\text{masked}} / \sqrt{d_k}$.

- **Why:** if Q/K components have roughly unit variance, a dot product's variance grows with d_k . Larger raw scores make softmax excessively peaked, producing tiny gradients and brittle early training.
- **Intuition:** divide by expected score scale so attention sharpness does not accidentally depend on head width.

14. Softmax

$\text{softmax}(s)_j = \exp(s_j) / \sum_r \exp(s_r)$.

- **Why probabilities/weights:** a convex mixture gives a stable differentiable routing mechanism and comparable relative allocation across sources.
- **Why raw scores cannot be used directly:** they are unbounded, signed, and not normalized; they would arbitrarily scale or subtract values.
- **Scores:** preference evidence. **Weights:** normalized allocation. Softmax is applied independently for each query row.

15. Weighted Sum of Values

$$\mathbf{O} = \mathbf{AV}, \text{ so } o_i = \sum_{j \leq i} a_{ij} v_j.$$

- **What:** matrix multiplication blending each allowed source's value according to that destination's attention weights.
- **Why blend, not copy:** relevance is usually distributed—syntax, entity attributes, and prior constraints can come from multiple places. A weighted mixture is differentiable and supports uncertainty.
- **Next:** one output vector per position/head is projected and merged into the residual stream.

16. Attention Output

- **Attention output** $\mathbf{O} \in \mathbb{R}^{(B \times T \times d_v)}$ **per head:** activation containing information read from the context for each position.
- **What it is not:** not the entire representation of a token; it omits the direct residual input and the FFN update.
- **Hidden state:** persistent layer-to-layer residual-stream vector. **Attention output:** ephemeral retrieval result inside one layer.

17. $\mathbf{WO}$

- **Output projection** $\mathbf{WO} \in \mathbb{R}^{(h \cdot d_v \times d)}$: learned parameter. $\mathbf{U} = \text{Concat}(\mathbf{O}^1, \dots, \mathbf{O}^h) \mathbf{WO} \in \mathbb{R}^{(B \times T \times d)}$.
- **Why:** heads use small specialist subspaces. Concatenation must be mixed and returned to the residual stream's common width d .
- **Multi-head intuition:** distinct heads can learn different relation types or routing patterns in parallel; not every head necessarily has an interpretable role.
- **Not LM head:** $\mathbf{WO}$ produces hidden-space updates for every token; $\mathbf{WLM}$ produces vocabulary scores from the final layer.

18. Residual Connections

$$\mathbf{R} = \mathbf{H} + \mathbf{U} \text{ (and later } \mathbf{R} = \mathbf{N} + \mathbf{F} \text{)}. \text{ Shapes match: all are } B \times T \times d.$$

- **Why add instead of replace:** preserve the current representation while letting a sublayer contribute an incremental correction.
- **Advantages:** direct gradient paths, easier optimization of deep stacks, and the option for a layer to learn a small/no-op update.
- **Next:** the sum is normalized before/after the following computation depending on architecture.

19. LayerNorm

$$\text{For one vector } \mathbf{r} \in \mathbb{R}^d: \mu = (1/d) \sum_k r_k, \sigma^2 = (1/d) \sum_k (r_k - \mu)^2; \text{LN}(\mathbf{r}) = \gamma \frac{(\mathbf{r} - \mu)}{\sqrt{(\sigma^2 + \epsilon)}} + \beta.$$

- $\gamma, \beta \in \mathbb{R}^d$: learned parameters; μ, σ^2 are temporary per-token statistics; ϵ is fixed numerical stabilizer.
- **Why:** keeps residual-stream feature scale controlled, improves conditioning and training stability, while γ/β let the model restore useful per-feature scale/offset.
- **Unlike BatchNorm:** normalizes across features of each token independently; sequence/batch length need not be fixed.

20. Feed Forward Network

$$\text{For each token independently: } \mathbf{F}(\mathbf{H}) = \mathbf{W} \phi(\mathbf{H}\mathbf{W} + \mathbf{b}) + \mathbf{b}.$$

Tensor	Shape	Type
W_{ff}	$d \times d_{\text{ff}}$	learned
b_{ff}	d_{ff}	learned (if present)
W_{out}	$d_{\text{ff}} \times d$	learned
b_{out}	d	learned (if present)

- **Why expand then compress:** d_{ff} (often $\sim 4d$) gives a larger feature workspace; compression writes the selected nonlinear features back to d .
- **Why nonlinear activation:** stacked linear maps collapse into one linear map. **ReLU**, **GELU**, or gated **SwiGLU** create conditional, richer transformations.
- **Attention vs FFN:** attention mixes information **between positions**; FFN transforms features **at each position**, using identical weights at all positions.
- **Next:** add its result to the attention-residual stream and normalize to form a new hidden state.

21. End of Layer

$H_{\text{end}} = \text{LN}(R + F(R)) \in \mathbb{R}^{(B \times T \times d)}$ in this post-LN schematic.

- **What:** new hidden states, one contextual vector per token.
- **Why:** combines preserved information, retrieved context, and nonlinear local computation.
- **Next:** exactly H_{end} —not $Q/K/V$ or weights—feeds layer $l+1$.

22. Layer 2

- **What:** same architecture and shapes as layer 1.
- **Why:** depth permits iterative contextual computation: later $Q/K/V$ are formed from already contextual hidden states.
- **Crucial:** it has independently learned WQ^2 , WK^2 , WV^2 , WO^2 , FFN weights, and LayerNorm parameters. Same blueprint $\neq$ same function.

23. LM Head (WLM)

- **LM head / output projection** $W_{\text{LM}} \in \mathbb{R}^{(d \times V)}$: learned parameter mapping final last-token state $h_{\text{last}} \in \mathbb{R}^d$ to $z = h_{\text{last}} W_{\text{LM}} + b_{\text{LM}} \in \mathbb{R}^V$; $b_{\text{LM}} \in \mathbb{R}^V$ is optional learned bias.
- **Why:** generation chooses among V discrete token classes, so hidden dimension must become vocabulary dimension.
- **WO vs W_{LM} :** $WO: h \cdot d_v \rightarrow d$, inside every layer, preserves working space. $W_{\text{LM}}: d \rightarrow V$, once at the end, produces class scores.
- **Weight tying:** many models set $W_{\text{LM}} = E_{\text{out}}$ (orientation-dependent notation), sharing input and output token representations. This reduces parameters and often improves learning; models may also leave them untied.

24. Logits

- **Logits** $z \in \mathbb{R}^V$: raw, unnormalized output scores—one per vocabulary token.
- **Why:** linear class scores retain signed confidence evidence and are convenient for training; softmax converts them to a normalized distribution only when needed.
- **Logits vs probabilities:** logits need not be positive or sum to one; probabilities are in $[0, 1]$ and sum to one.

25. Final Softmax

$$p_i = \exp(z_i) / \sum_{r=1}^V \exp(z_r).$$

- **Why:** turns V competing logits into a categorical next-token distribution for loss calculation or sampling.
- **Next:** a decoding rule chooses one token ID.

26. Token Selection

- **Greedy:** choose $\text{argmax}_i p_i$; deterministic, can be repetitive.
- **Sampling:** draw from p ; adds variety.
- **Temperature τ :** use $\text{softmax}(z/\tau)$; lower <1 sharpens, higher >1 flattens.
- **Top-k:** retain only the k highest-logit tokens, renormalize.
- **Top-p (nucleus):** retain smallest set with cumulative probability $\geq p$, renormalize.
- **Why:** model distribution is not itself a complete product decision; decoding controls the quality/diversity trade-off.

27. Autoregressive Generation

prefix $\rightarrow$ all hidden states $\rightarrow$ last-position state $\rightarrow$ next-token distribution
 $\rightarrow$ choose ID $\rightarrow$ append to prefix $\rightarrow$ repeat

- **Why only last hidden state predicts:** after prefix $[x_1, \dots, x_T]$, position T has access to the complete prefix and is trained to predict x_{T+1} .
- **Why every token is still updated:** last state needs context; attention builds that context from updated states at all earlier positions. During training, every position supplies a parallel next-token prediction.

28. KV Cache

- **Without cache:** generating each new token recomputes K and V for every old token in every layer—wasteful.
- **With cache:** for layer l , store old $K_{\text{cache}}, V_{\text{cache}} \in \mathbb{R}^{(B \times T_{\text{old}} \times d_k/d_v)}$. On one new token, compute only $q_{\text{new}}, k_{\text{new}}, v_{\text{new}}$; append K/V to cache; q_{new} attends over cached + new keys/values.
- **Why only new Q :** old positions cannot see the future under a causal mask, so their old outputs do not change when a token is appended. The new position is the only one needing a new context read.
- **Cache contains:** per-layer keys and values, not attention weights and not usually queries.

29. Complete Worked Example

Deliberately tiny illustrative model: $V=5$, $d=d_k=d_v=2$, $T=3$, one head, two layers, no biases, post-LN, $\epsilon=0$, FFN is ReLU with $d_{\text{ff}}=2$. This makes arithmetic visible, not realistic. Vocabulary: $[\text{The}:0, \text{cat}:1, \text{sat}:2, \text{on}:3, \text{mat}:4]$; input IDs $[0, 1, 2] = \text{"The cat sat"}$.

$$E \in \mathbb{R}^{(5 \times 2)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ -1 & 0 \\ 0 & -1 \end{bmatrix}; P \in \mathbb{R}^{(3 \times 2)} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}.$$

$$H = E[x] + P = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \text{ (rows: The, cat, sat)}.$$

Layer 1

Parameters: $WQ^1 = WK^1 = WV^1 = W0^1 = I$; $W^1 = I$; $W^1 = I$; LayerNorm $\gamma = [1, 1]$, $\beta = [0, 0]$.

Q/K/V (each 3×2 activation):

$$Q^1 = H \blacksquare WQ^1 = \begin{bmatrix} 1, 0 \\ 0, 1 \\ 1, 1 \end{bmatrix} \quad \begin{bmatrix} 1, 0 \\ 0, 1 \end{bmatrix} = \begin{bmatrix} 1, 0 \\ 0, 1 \\ 1, 1 \end{bmatrix}.$$

$$K^1 = H \blacksquare WK^1 = \begin{bmatrix} 1, 0 \\ 0, 1 \\ 1, 1 \end{bmatrix}; \quad V^1 = H \blacksquare WV^1 = \begin{bmatrix} 1, 0 \\ 0, 1 \\ 1, 1 \end{bmatrix}.$$

Scores: $S^1 = Q^1 (K^1) \blacksquare = \begin{bmatrix} 1, 0, 1 \\ 0, 1, 1 \\ 1, 1, 2 \end{bmatrix}.$

Mask then scale ($\sqrt{d_k} = \sqrt{2} \approx 1.414$):

$$S_{\text{masked}}^1 = \begin{bmatrix} 1, -\infty, -\infty \\ 0, 1, -\infty \\ 1, 1, 2 \end{bmatrix};$$

$$S_{\text{scaled}}^1 \approx \begin{bmatrix} 0.707, -\infty, -\infty \\ 0, 0.707, -\infty \\ 0.707, 0.707, 1.414 \end{bmatrix}.$$

Row softmax: $A^1 \approx \begin{bmatrix} 1, 0, 0 \\ 0.330, 0.670, 0 \\ 0.248, 0.248, 0.503 \end{bmatrix}.$

Weighted values:

$$O^1 = A^1 V^1 \approx \begin{bmatrix} 1, 0 \\ 0.330 \cdot 1, 0 + 0.670 \cdot 0, 1 \\ 0.248 \cdot 1, 0 + 0.248 \cdot 0, 1 + 0.503 \cdot 1, 1 \end{bmatrix}$$

$$O^1 \approx \begin{bmatrix} 1, 0 \\ 0.330, 0.670 \\ 0.751, 0.751 \end{bmatrix}.$$

W0, residual, LN: $U^1 = O^1 W_0^1 = O^1; R^1 = H \blacksquare U^1 \approx \begin{bmatrix} 2, 0 \\ 0.330, 1.670 \\ 1.751, 1.751 \end{bmatrix}.$

For 2 features, LN maps any unequal $[a, b]$ to $[1, -1]$ if $a > b$, and $[-1, 1]$ if $a < b$; equal features map to $[0, 0]$ (with the stated $\epsilon=0$ convention understood as limiting case). Thus $N^1 = \text{LN}(R^1) = \begin{bmatrix} 1, -1 \\ -1, 1 \\ 0, 0 \end{bmatrix}.$

FFN: $G^1 = N^1 W^1 = N^1; \text{ReLU}(G^1) = \begin{bmatrix} 1, 0 \\ 0, 1 \\ 0, 0 \end{bmatrix}; F^1 = \text{ReLU}(G^1) W^1 = \begin{bmatrix} 1, 0 \\ 0, 1 \\ 0, 0 \end{bmatrix}.$

Final residual + LN: $R_{\text{ff}}^1 = N^1 + F^1 = \begin{bmatrix} 2, -1 \\ -1, 2 \\ 0, 0 \end{bmatrix}; H^1 = \text{LN}(R_{\text{ff}}^1) = \begin{bmatrix} 1, -1 \\ -1, 1 \\ 0, 0 \end{bmatrix}.$

Layer 2

Parameters again are identity matrices and $\gamma = [1, 1], \beta = [0, 0]$ only to keep the worked arithmetic compact; in a real model these are separate learned tensors.

$$Q^2 = K^2 = V^2 = H^1 = \begin{bmatrix} 1, -1 \\ -1, 1 \\ 0, 0 \end{bmatrix}.$$

$$S^2 = Q^2 (K^2) \blacksquare = \begin{bmatrix} 2, -2, 0 \\ -2, 2, 0 \\ 0, 0, 0 \end{bmatrix}.$$

$$S_{\text{scaled}}^2 \approx \begin{bmatrix} 1.414, -\infty, -\infty \\ -1.414, 1.414, -\infty \\ 0, 0, 0 \end{bmatrix} \text{ after causal masking/scaling.}$$

$$A^2 \approx \begin{bmatrix} 1, 0, 0 \\ 0.056, 0.944, 0 \\ 0.333, 0.333, 0.333 \end{bmatrix}.$$

$$O^2 = A^2 V^2 \approx \begin{bmatrix} 1, -1 \\ 0.056 \cdot 1, -1 + 0.944 \cdot -1, 1 \\ 0, 0 \end{bmatrix} \approx \begin{bmatrix} 1, -1 \\ -0.888, 0.888 \\ 0, 0 \end{bmatrix}.$$

$$U^2 = O^2; R^2 = H^1 + U^2 \approx \begin{bmatrix} 2, -2 \\ -1.888, 1.888 \\ 0, 0 \end{bmatrix}; N^2 = \text{LN}(R^2) = \begin{bmatrix} 1, -1 \\ -1, 1 \\ 0, 0 \end{bmatrix}.$$

$$F^2 = \text{ReLU}(N^2) = \begin{bmatrix} 1, 0 \\ 0, 1 \\ 0, 0 \end{bmatrix}; H^2 = \text{LN}(N^2 + F^2) = \begin{bmatrix} 1, -1 \\ -1, 1 \\ 0, 0 \end{bmatrix}.$$

LM head and prediction: last state $h_{\text{last}} = [0, 0]$. Let

$$W_{\text{LM}} \in \mathbb{R}^{(2 \times 5)} = \begin{bmatrix} 1, 0, -1, 0, 1 \\ 0, 1, 0, -1, -1 \end{bmatrix}.$$

$$z = h_{\text{last}} W_{\text{LM}} = [0, 0] W_{\text{LM}} = [0, 0, 0, 0, 0]; \text{softmax}(z) = [0.2, 0.2, 0.2, 0.2, 0.2].$$

This toy's final state is deliberately degenerate because of symmetric choices. Greedy has a five-way tie; choose a tie rule or sample. **Engineering lesson:** all calculations follow the real pipeline; trained non-symmetric weights create useful logits. If $h_{\text{last}} = [1, -1]$ instead, $z = [1, -1, -1, 1, 2]$, $\text{softmax} \approx [0.207, 0.028, 0.028, 0.207, 0.563]$, predicting `mat` greedily.

30. Common Beginner Confusions

Question	Revision answer
Embedding matrix vs input embeddings	E is learned $V \times d$ table; input embeddings are selected $T \times d$ rows.
Embedding vs hidden state	Lookup is context-free; hidden state is contextual layer activation.
Hidden state vs attention output	Hidden state survives layers; attention output is one temporary retrieval contribution.
Scores vs weights	Scores are raw $Q \cdot K$ compatibility; weights are masked/scaled softmax scores.
Weights vs output	Weights say where to read; output is weighted values—the content read.
Query vs key	Query requests; key advertises matchability.
Key vs value	Key chooses relevance; value supplies payload.
W_O vs W_LM	W_O returns attention to hidden width; W_LM maps hidden width to vocabulary.
FFN vs attention	FFN mixes features within one position; attention mixes information across positions.
Training vs inference	Training updates parameters using targets; inference freezes them and generates.
Parameters vs activations	Parameters persist/learn; activations are computed per prompt.
Layer vs attention head	Layer is full attention+FFN block; head is one parallel attention subspace inside it.
Attention layer vs Transformer layer	Attention is a sublayer; Transformer layer also includes residuals, norms, FFN.
Logits vs probabilities	Logits are raw class scores; probabilities are normalized softmax values.
Attention softmax vs LM softmax	Attention normalizes over source positions per query; LM softmax over vocabulary tokens.
Why only last token predicts	It has the full prefix and corresponds to the next unfilled position.
Why every token updates	Last token's context depends on representations of earlier tokens.
Why every layer has different W_Q	Each layer needs a new learned relation test over richer states.
Why $Q/K/V$ disappear	They are layer-local routing activations; only H_{next} is next layer's state.
What is passed next	New hidden states H_{next} , plus cached K/V only during incremental inference.
Why residuals	Preserve signal and make deep optimization feasible.
Why LayerNorm	Stabilize feature scale and optimization.
Why FFN	Add nonlinear per-token feature computation.
Why output projection	Merge heads back to the shared residual stream.
Why LM head	Convert d features into V token choices.
Vocabulary size vs embedding size	V = number of token types/classes; d = vector feature width.
Hidden vs vocabulary dimension	Hidden dimension is internal workspace; vocabulary dimension is final number of possible tokens.

31. One-Page Summary

Text $\rightarrow$ tokenizer $\rightarrow$ IDs $\times (B \times T)$
 $\rightarrow E[x] + P = H_{\text{next}} (B \times T \times d)$
 $\rightarrow$ repeat L times:
 $Q = HWQ, K = HWK, V = HWV$
 $A = \text{softmax}((QK^T + \text{causal mask})/\sqrt{d_k})$

```

    U=Concat(AV across heads)W0
    N=LN(H+U)
    H=LN(N + W $\phi$ (NW $\phi$ +b $\phi$ )+b $\phi$ )
→ h_last = H[:,T,:] (d)
→ logits z=h_last WLM+bLM (V)
→ p=softmax(z) (V)
→ select token ID → append → repeat

```

- $E, P, WQ, WK, WV, W0, W\phi, W\phi, \gamma, \beta, WLM$ (and biases if used) are learned parameters: update in training, frozen in inference.
- $H, Q, K, V, S, A, O, U, N, z, p$ are activations: recomputed from the prompt. K/V can be retained in an inference KV cache.
- Attention answers **where should this position read?**; values provide **what it reads**; FFN answers **how should this position transform its features?**
- Causal masking enforces: position i sees only $\leq i$. The final position therefore predicts the next token without seeing it.